property cannot be found, you return a `ValidationResult` with an error message, which is stored in `ModelState`. If it is found, the first letters of each property are compared. Then you either return a `ValidationResult` with a different error message if the validation fails, or you return `ValidationResult.Success`.

Now that the attribute has been created, you can remove the original check in the `OnPost` method—the lines of code before checking `ModelState.IsValid` in listing 5.29. Then you apply the attribute to the target property, which is the `CountryName` property of the `InputModel`:

```
[Required, CompareFirstLetter(OtherProperty = nameof(CountryCode))]
public string CountryName { get; set; }
```

The custom attribute is executed along with framework attributes after model binding.

In addition to writing your own validation logic, whether inline within handlers or as a custom attribute, you can also consider using `IValidatableObject` (http://mng.bz/XZRv) for more complex validation requirements. Alternatively, you can explore some third-party validation libraries. One particularly popular open source library is Fluent Validation (https://fluentvalidation.net/). It offers much more flexibility than the validation attributes provided by various parts of .NET.

This has been an in-depth chapter, but we are only halfway through working with forms in a Razor Pages application. We have touched on using some form-related tag helpers so far, but we have barely scratched the surface. In the next chapter, you continue your journey by taking a deeper dive into more tag helpers that ease the burden of working with forms in Razor Pages.